From RAG to Agentic RAG

Explore the evolution of Retrieval-Augmented Generation into agentic RAG. Understand how standard RAG grounds large language models with external data, and discover the need for agentic workflows that add reasoning, planning, and tool integration to solve complex queries effectively.

Welcome to the first lesson of our course on Agentic RAG! In this lesson, we will build the essential foundation for everything that follows. Before we can run, we need to learn how to walk. In the world of advanced AI, that means getting a rock-solid understanding of a revolutionary technique called Retrieval-Augmented Generation, or RAG.

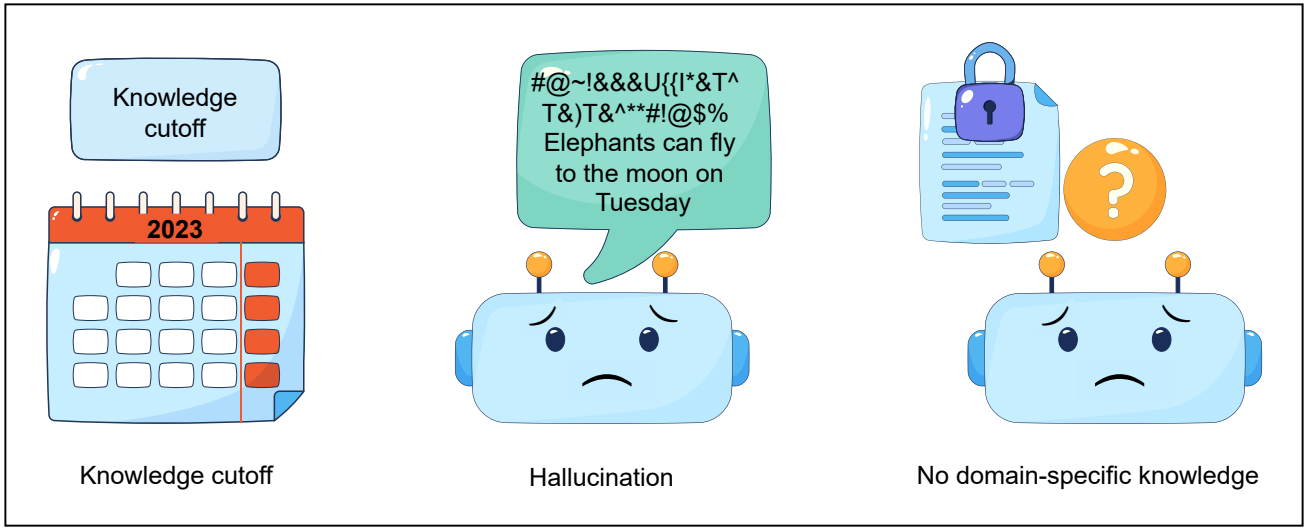
By the end of this lesson, you'll not only understand what RAG is and why it's so important, but you'll also begin to see its limitations. You'll learn to appreciate why the next evolution, the 'agentic' approach, is creating so much excitement in the field.

Let's begin.

What is RAG, and why do we need it?

If you've spent any time with large language models (LLMs) like OpenAI's GPT series, Meta's Llama models, or Google's Gemini family, you know they are incredibly powerful. They can write poetry, debug code, and explain

complex topics with clarity. However, you've also likely run into their fundamental limitations:



Limitations of LLMs

- **The knowledge cutoff:** LLMs are static snapshots in time. Their knowledge is frozen at the point where their training data was first collected. Ask a model about an event that happened yesterday, and it will often reply with something like, "I only have information up to early 2023."
- **Hallucinations:** When an LLM doesn't know the answer to a specific question, it doesn't always admit it. Instead, it can generate a plausible-sounding answer that is completely wrong. This phenomenon, known as **hallucination**, is a major barrier to using LLMs in professional, fact-sensitive applications.
- **Lack of domain-specificity:** You can't expect a general-purpose model to have deep, internal knowledge of your company's private financial reports, your legal team's case files, or your project's technical documentation.

Ask

These problems severely limit the reliability and usefulness of LLMs in real-world business contexts. This is precisely the problem that Retrieval-Augmented Generation (RAG) was designed to solve.

The core idea behind RAG is simple yet profound:

> Don’t force the LLM to answer from memory. Instead, give it an “open-book exam.”

Before prompting the LLM to answer a question, a RAG system first retrieves relevant, up-to-date information from an external knowledge source. It then provides this information to the LLM as context, along with the user’s original question. This grounds the model in reality, dramatically reducing hallucinations and allowing it to answer questions using information it was never trained on.

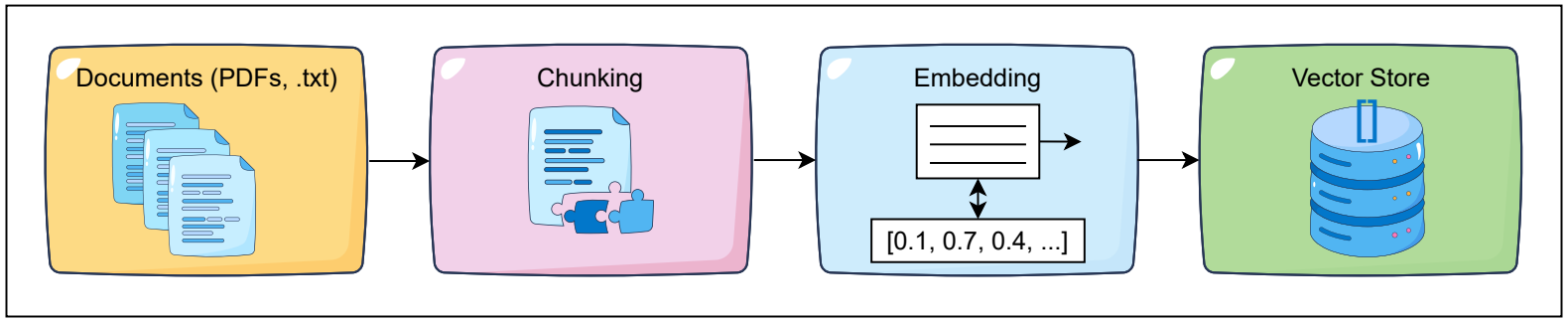
Components and workflow of a RAG System

A RAG system isn’t a single, monolithic thing. It’s a two-stage process. First, we prepare our knowledge base in a stage called **indexing**. This is usually done offline. Second, we use that prepared knowledge base to answer questions in real-time in the retrieval and generation stage.

Stage 1: Indexing (Offline) and preparing the knowledge base

This is the preparatory work needed to make your data searchable.

Ask



The offline indexing workflow

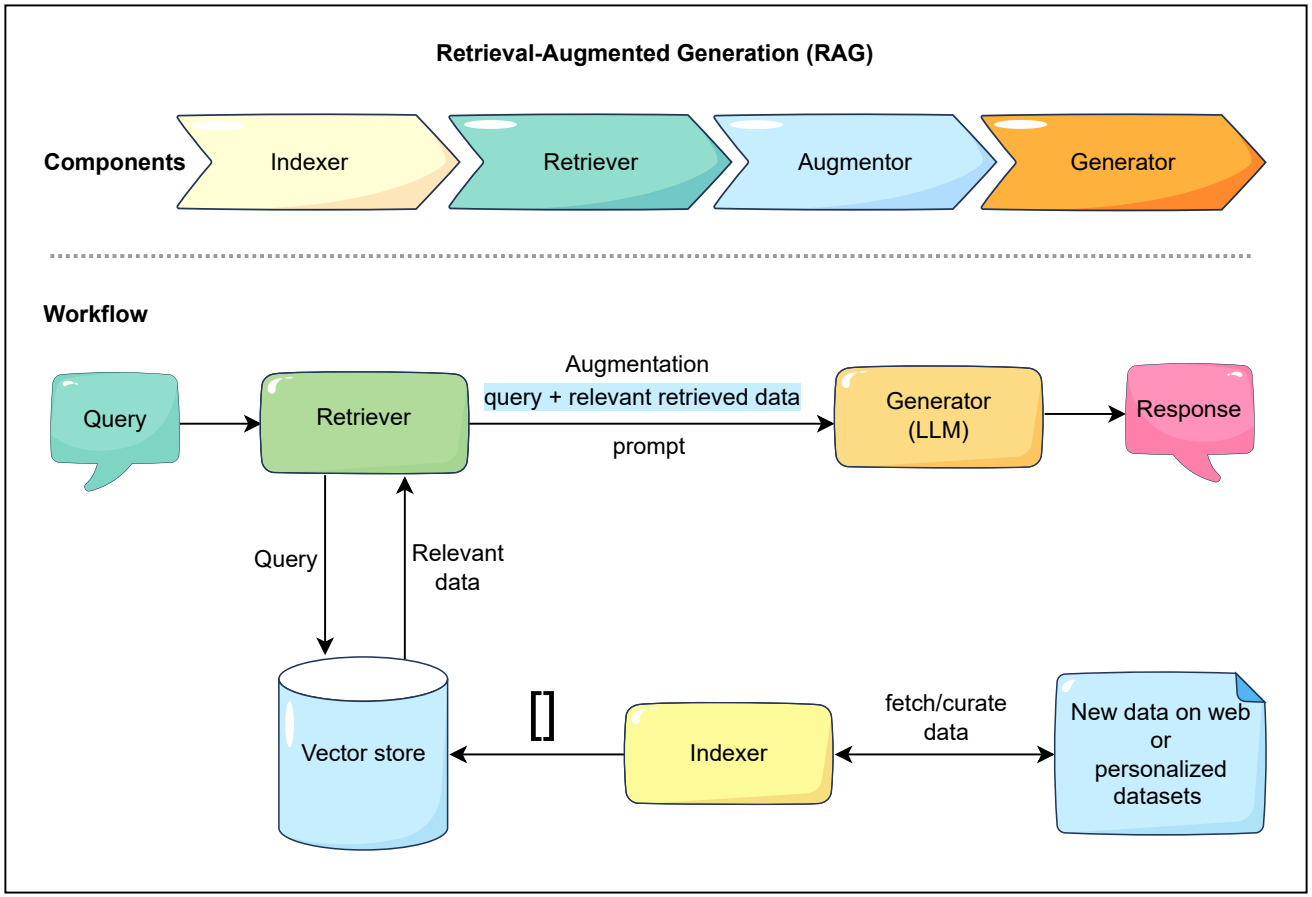
1. **Load data:** The system ingests your documents from various sources. These can be PDFs, text files, web pages, Notion databases, or any other data you want the model to know about.
2. **Chunk/split:** Documents are often too long to fit into an LLM's context window. So, we break them down into smaller, manageable pieces, or "chunks." This is crucial because it allows the system to find highly specific and relevant pieces of information instead of entire, noisy documents.
3. **Embed:** This is where everything falls into place. Each text chunk is fed into an embedding model (like the efficient `all-MiniLM-L6-v2`, OpenAI's more recent `text-embedding-3-small`, or locally run models like `nomic-embed-text` which we'll use later in this course). This model converts the semantic meaning of the text into a numerical list called a **vector**. Chunks with similar meanings will have similar vector values.
4. **Store:** These numerical vectors, along with the original text chunks they represent, are loaded into a specialized vector store or vector database. This database is highly optimized for finding vectors that are "closest," or most similar to a given query vector.

Ask

Stage 2: Retrieval, generation (online) and answering the user's query

This is the real-time process that happens every time a user asks a question. As we can see in the diagram below, the flow is clear and linear. Data is processed and stored offline. When a query comes in, we retrieve

relevant data, augment the prompt, and then generate the final, grounded answer.



The RAG workflow: Offline indexing prepares data in a vector store. Online retrieval uses the query to fetch relevant data, augment the prompt, and generate a grounded response via an LLM.

1. **User query:** The process begins when a user submits their question (e.g., “What is our company’s p
remote work?”).

Ask

2. **Embed query:** The same embedding model used in the indexing stage converts the user's query into a vector.
3. **Search/retrieve:** The system takes the query vector and uses it to search the vector store. It performs a similarity search to find the text chunks whose vectors are mathematically closest to the query vector. These are the chunks most semantically relevant to the user's question.
4. **Augment prompt:** The system then takes the most relevant retrieved chunks (the "context") and dynamically inserts them into a prompt template alongside the user's original query.
5. **Generate response:** Finally, this complete, context-rich prompt is sent to the LLM. The model is instructed to formulate its answer based *only* on the provided context. This forces the LLM to act as a "reasoning engine" over the data you provided, rather than trying to answer from its internal memory.

RAG in action: A practical example

Let's make this concrete with a simple scenario.

- **Scenario:** We've built an HR chatbot for our company.
- **Knowledge base:** We've indexed all our HR policy PDFs into a vector store.
- **User query:** "How many days of paid time off do new employees get per year?"

Here's how the RAG workflow handles this:

1. The query, "How many days of paid time off do new employees get per ye
2. The system searches the vector store of HR policy chunks.
3. It retrieves the top 3 most relevant chunks. Let's say it finds the ones mentioned below.

Ask

I. **Chunk 1 (from pto-policy.pdf):** “Policy 4.1a: New employees joining the company are entitled to 20 days of paid time off (PTO) annually. This is in addition to public holidays.”

II. **Chunk 2 (from pto-policy.pdf):** “PTO accrues on a bi-weekly basis and is available for use after the 90-day probationary period.”

III. **Chunk 3 (from employee-handbook.pdf):** “Requesting time off must be done through the employee portal at least two weeks in advance.”

4. These chunks are automatically formatted into a prompt sent to the LLM:

```
Context:
- "Policy 4.1a: New employees joining the company are entitled to 20 days of paid time off (PTO) annually. Thi
- "PTO accrues on a bi-weekly basis and is available for use after the 90-day probationary period."
- "Requesting time off must be done through the employee portal at least two weeks in advance..."

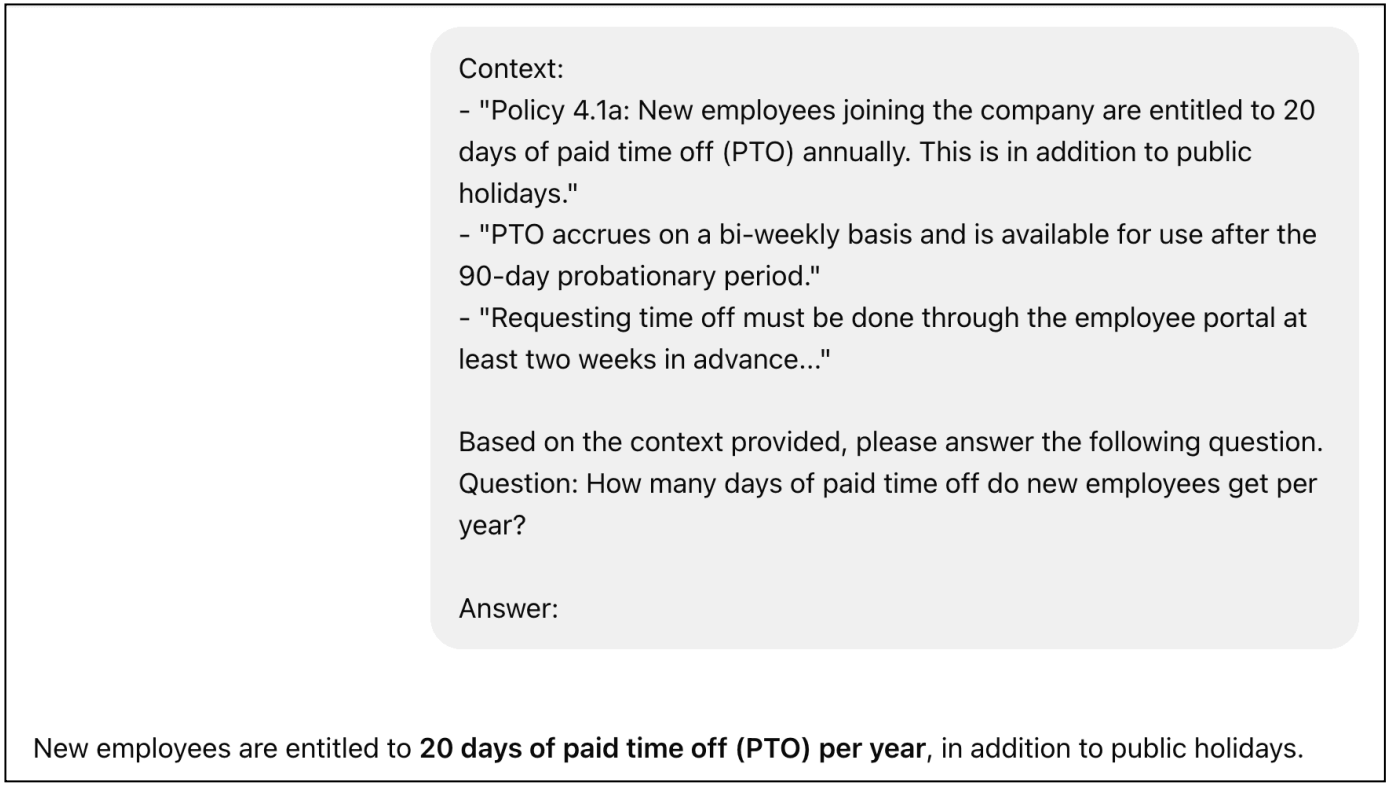
Based on the context provided, please answer the following question.
Question: How many days of paid time off do new employees get per year?

Answer:
```

Augmented prompt structure: The retrieved context and user’s question are combined, instructing the LLM to answer based only on the provided facts

5. **Final, grounded answer from the LLM:** “Based on the policy, new employees get 20 days of paid time off (PTO) per year.”

Ask



Example augmented prompt: The combined context and question guide the LLM to generate an answer grounded in the retrieved policy details

As we can see, the final answer is accurate, factual, and directly sourced from the company's own documents. This is the power of RAG. However, as we're about to see, this relatively straightforward pipeline also has its limits.

From static pipeline to agentic workflow

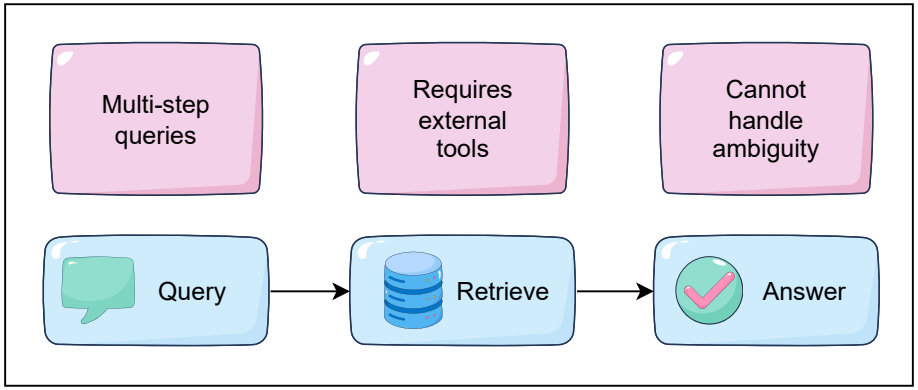
Ask

The RAG system we've just described is incredibly effective for direct question-answering. It's a massive leap forward from using an LLM on its own. However, its strength, its simple, predictable, linear process, is also its

greatest weakness. Let's explore where this rigid pipeline begins to fall apart.

Where standard RAG struggles

A static RAG pipeline follows the exact same `retrieve -> augment -> generate` workflow every single time, regardless of the user's query. This works perfectly for simple questions but fails when a query requires reasoning, decomposition, or access to the outside world.



- **Limitation 1: Complex, multi-step queries**
  - **Example:** "Compare the pros and cons of our company's PPO and HMO health insurance plans based on the benefits PDFs."
  - **Why it fails:** Standard RAG doesn't "think." It will see the terms "PPO," "HMO," and "compare" and retrieve a mix of chunks from both documents. It then crams this jumbled context into a single prompt. The LLM is forced to sift through the combined text and often produces a poor, unstructured su instead of a neat, side by side comparison. It cannot formulate a plan like, "First, I'll get the PPO information. Second, I'll get the HMO information. Third, I will compare them."

Ask

- **Limitation 2: The need for external tools**
  - **Example:** “Summarize the latest Q3 earnings report PDF and tell me how our company’s current stock price is performing today.”
  - **Why it fails:** The RAG pipeline can flawlessly retrieve and summarize the earnings report because it’s in the knowledge base. However, the system is a closed world. It has no mechanism to access a real-time stock price API or browse the web. The pipeline is fixed and has no “tool slots” to call upon other resources.
- **Limitation 3: Ambiguity and disambiguation**
  - **Example:** “Tell me about our ‘Titan’ project.”
  - **Why it fails:** Imagine your company has three projects named “Titan.” One in is engineering, a past one is from marketing, and a new initiative is in finance. A static RAG pipeline will simply find the chunks that are mathematically most similar to the query “Titan project.” It might pick the wrong one, or a mix of all three, leading to a confusing answer. It has no ability to pause and ask the user a clarifying question like, “Do you mean the engineering, marketing, or finance ‘Titan’ project?”

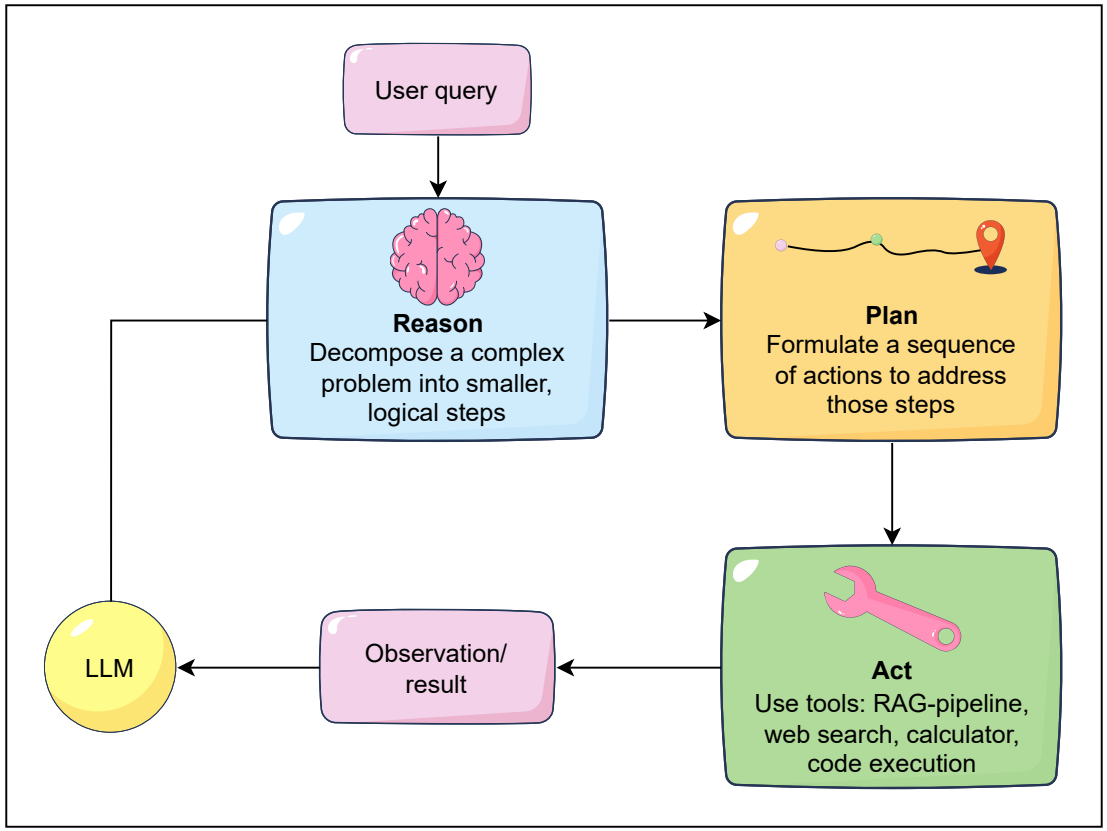
The solution: Introducing “agency”

To overcome these limitations, we need to move beyond a fixed pipeline. We need a system that can reason, plan, and adapt its strategy based on the query. We need to introduce agency. Crucially, agents aren’t intended to *replace* RAG; they learn to *orchestrate* RAG, treating the entire retrieval and generation pipeline as a powerful tool within a larger workflow.

Ask

An **AI agent** is a system that uses an LLM not just to generate text, but as a reasoning engine to direct its own actions. A true agent can perform the actions outlined below.

1. **Reason:** Decompose a complex problem into a series of smaller, logical steps.
2. **Plan:** Formulate a sequence of actions required to address those steps.
3. **Use tools:** Select and execute actions, using a variety of available resources, like a RAG pipeline, a web search, a calculator, or code execution.



The core agentic loop: An agent uses its LLM brain to reason (decompose the problem), plan (formulate actions), and act (use tool
iteratively refining based on observations

Ask

This leads to a powerful analogy that perfectly captures the shift.

Standard RAG is a fast librarian: You ask for information, and it quickly fetches the most relevant documents for you to read.

An agent is a skilled research assistant: You give it a complex research question. It might first visit the library (use the RAG tool), then check today's news (use a web search tool), then perform some calculations (use a calculator tool). Finally, it will synthesize all of its findings into a complete, coherent answer.

A glimpse into agentic RAG

In an agentic RAG system, the RAG pipeline we've learned about is no longer the entire workflow. Instead, it becomes just one powerful tool in an agent's toolbox.

Let's revisit our "compare health plans" example and see how an agent would tackle it.

- **User query:** "Compare the pros and cons of our company's PPO and HMO health insurance plans."

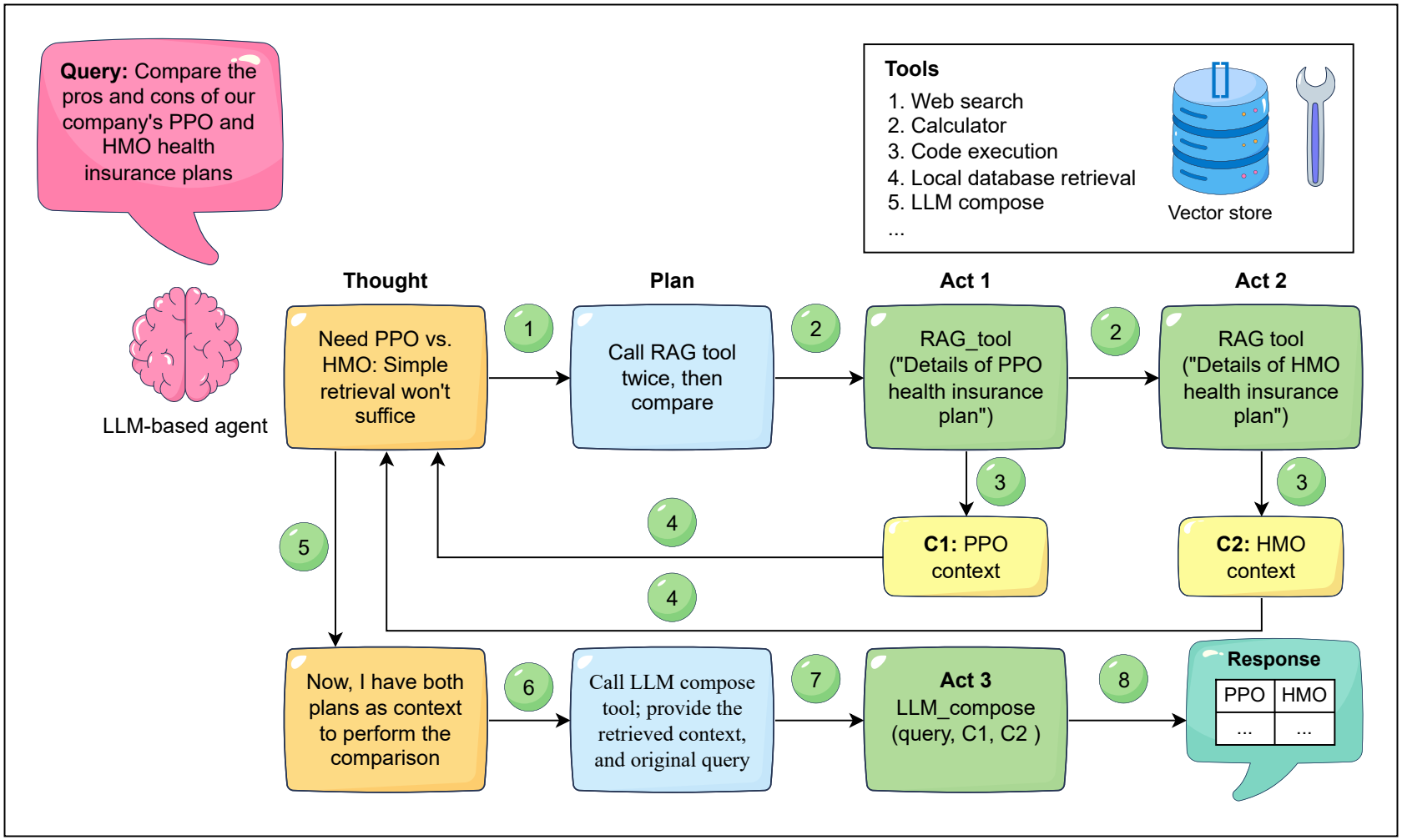
The agent, powered by a reasoning LLM, would initiate a dynamic, multi-step thought process.

1. **Thought:** "The user wants a comparison of two distinct items: PPO and HMO plans. A simple retrieval won't be enough. I need to gather information on each one separately and then combine them. I will use my RAG tool twice."
2. **Action (Step 1):** Use the RAG tool with the specific query, "Details of the PPO health insurance plan."
    I. **Result:** Retrieves context C1 about the PPO plan.

Ask

3. **Action (Step 2):** Use the RAG tool again with the specific query, “Details of the HMO health insurance plan.”
    I. **Result:** Retrieves context C2 about the HMO plan.
4. **Thought:** “I now have structured information for both plans. I have everything I need to perform the final comparison.”
5. **Action (Step 3):** Pass both contexts (C1 and C2) to the LLM with a final instruction, “Based on the following information, create a side by side comparison of the pros and cons of the PPO and HMO plans.”

Ask



Agentic workflow example: The agent plans and executes multiple RAG tool calls sequentially (Act 1, Act 2) based on its reasoning, using observations (C1, C2) to synthesize the final comparison (Act 3)

This is a fundamentally different, and more intelligent process. The agent dynamically planned and executed a series of steps to fulfill the user's complex request.

Ask

Test your understanding

You’ve built a RAG-based internal assistant for a law firm. Lawyers ask, “Summarize the most recent case law about data privacy in 2025.” The model retrieves 3 older documents from 2023, but misses the newest case and cannot clarify user intent.

Question: How would you extend this system into an agentic RAG design? Identify which agent capabilities (reasoning, planning, tool use) would solve the problem, and how you’d integrate them. Detail your solution and reasoning.

Evaluate your answer!

